

of C++. Some of the important features introduced by C++98 include changes in type casting, introduction of Boolean operators and mutable class members, etc. There were modifications in the C++ Standard Library also. Though not explicitly mentioned in the official documents, the Standard Template Library (STL) offering algorithms, containers, functions, and iterators also influenced the C++98 standard. I believe that until the arrival of STL, most C++ programs could have been ported to C without much difficulty. But STL became a game changer and made C++ truly different from C.

Let us now focus on the g++ compiler. What is the default standard of the g++ compiler — C++98, C++03, C++11, C++14, C++17 or C++20? The answer is ‘None of these’. The default standard of g++ is C++XX with additional GNU specific features. Here, XX denotes a C++ standard and it depends on the version of GCC on your system. I have GCC 7.5.0 in my system and g++ defaults to C++14 with additional GNU specific features. But if you want to use a different standard, use the g++ option ‘-std=c++XX’ (XX denotes a specific standard of C++) which will compile the program by strictly following the rules set by the particular standard specified. For example, ‘g++ -std=c++98 pgm1.cc’ will compile the program *pgm1.cc* conforming to the C++98 standard. Similarly, the g++ option ‘-std=gnu++XX’ (XX denotes a specific standard of C++) will compile the program by the rules set by the particular standard specified along with GNU specific extensions. For example, ‘g++ -std=c++11 pgm1.cc’ will compile the program *pgm1.cc* conforming to the C++11 standard with GNU specific extensions. Now, consider the program *pgm2.cc* given below:

```
#include<iostream>
using namespace std;
int main( )
```

```
{
    int constexpr=98;
    cout<<"I am C++"<<constexpr<<endl;
    return 0;
}
```

The compilation and execution steps of the program *pgm2.cc* conforming to the C++98 standard are given below:

```
>> g++ -std=c++98 pgm2.cc
>> ./a.out
I am C++98
>> g++ -std=gnu++98 pgm2.cc
>> ./a.out
I am C++98
>> g++ -std=c++11 pgm2.cc
pgm2.cc: In function 'int main()':
pgm2.cc:5:15: error: expected
unqualified-id before '=' token
    int constexpr=98;
                ^
pgm2.cc:6:20: error: expected primary-
expression before 'constexpr'
    cout<<"I am C++"<<constexpr<<endl;
                    ^~~~~~
```

As you can see, the program successfully compiles and the message *I am C++98* gets printed on the screen. We can also see that the g++ option ‘-std=gnu++98’ also results in the same output. But we can observe that the same program shows errors when compiled as a program conforming to the C++11 standard. Could you explain the reason?

C++03 standard

C++03 (official document ISO/IEC 14882:2003) was introduced in 2003. Compared to the previous version of C++, C++03 only introduced minor changes to the language. This also reflects the way C++ standards are released, in general. The release of every major C++ standard is followed by the release of a minor C++ standard. This minor standard is released mainly to address the shortcomings of the previous major release as well as for

fixing bugs. No keywords were added or deprecated by C++03. The only new language feature added was value initialisation. This is the initialisation performed when an object is constructed with an empty initialiser. It is possible to use the g++ option ‘-std=c++03’ to make sure that a program conforms to the C++03 standard.

C++11 standard

C++11 (official document ISO/IEC 14882:2011) was introduced in 2011. C++11 is a major version of C++, and just like C++98 brought in a considerable amount of changes to the language. We will start our discussion by analysing the keywords. There are 73 keywords in C++11. The following 10 new keywords were added to the existing 63 keywords from C++98. They are *alignas*, *alignof*, *constexpr*, *char16_t*, *char32_t*, *decltype*, *noexcept*, *nullptr*, *static_assert* and *thread_local*.

Now, I hope you are able to explain the reason for the error shown in the output of the program *pgm2.cc* shown earlier. In C++11, *constexpr* is a keyword, whereas in C++98 it is not. The keyword *constexpr* allows functions to return constant expressions, which at compile time will lead to an optimisation whereby the function call is replaced with a constant value. C++11 also brought in lambda functions, thereby giving functional programming capabilities to C++. Thus C++ became a multi-paradigm programming language, at least to some extent. The keyword *export* was used to allow multiple declarations of templates in C++98 and C++03. In C++11, *export* was unused and reserved for future use. C++11 made several changes to the C++ Standard Library, and also introduced range-based *for* loops. Consider the program *pgm3.cc* given below:

```
#include<iostream>
using namespace std;
int main( )
{
```